

Problem A. Namomo Subsequence

The pattern `namomo` has equality type

$$A\ B\ C\ D\ C\ D,$$

where the four chosen characters are pairwise distinct. The alphabet has only 62 symbols, so we can scan the string and keep dynamic programming tables indexed by the two characters that will play the roles of C and D .

Maintain the following quantities for ordered distinct characters (c, d) :

- $p[c][d]$: the number of already chosen prefixes A, B, C with $C = c$ and future $D = d$;
- $q[c][d]$: the number of prefixes A, B, C, D ;
- $r[c][d]$: the number of prefixes A, B, C, D, C .

When the current character is x , process the transitions in reverse order:

1. finish the pattern by adding $r[c][x]$ to the answer for all $c \neq x$;
2. extend A, B, C, D to A, B, C, D, C by adding $q[x][d]$ to $r[x][d]$;
3. extend A, B, C to A, B, C, D by adding $p[c][x]$ to $q[c][x]$;
4. create A, B, C with $C = x$ and each $d \neq x$.

For the last operation we need the number of ordered prefix pairs (A, B) whose two characters are distinct and are both different from x and d . Keep $\text{cnt}[u]$, the number of prefix occurrences of character u , and $\text{pair}[u][v]$, the number of ordered prefix pairs (u, v) with $u \neq v$. If total is the total number of ordered distinct pairs in the prefix and $\text{involving}[u]$ is the number of such pairs using u , then

$$\#(A, B \notin \{x, d\}) = \text{total} - \text{involving}[x] - \text{involving}[d] + \text{pair}[x][d] + \text{pair}[d][x].$$

All additions are done modulo 998244353.

Complexity. $O(62n)$ time and $O(62^2)$ memory.

Problem B. Rectangle Flip 2

Give each cell a deletion time. A rectangle disappears exactly when the first cell inside it is deleted. Thus, if we can count for every time t the number of rectangles whose minimum deletion time is t , then the number of intact rectangles after each second follows by subtracting these counts from the total number of rectangles.

Assume $n \leq m$; otherwise transpose the board. Fix the top and bottom rows of a rectangle. For every column j , let

$$v_j = \min_{\text{row between top and bottom}} \text{time}[\text{row}][j].$$

A rectangle with these fixed top and bottom rows corresponds to a subarray of v , and its first deleted cell is the minimum element of this subarray. Since all deletion times are distinct, the minimum is unique.

For the current pair of rows, use a monotone stack to compute, for every column j , the number of subarrays for which v_j is the minimum. Add that number to $\text{cnt}[v_j]$. The array v can be updated incrementally when the bottom row is extended.

Let

$$R_0 = \frac{n(n+1)}{2} \cdot \frac{m(m+1)}{2}$$

be the initial number of rectangles. After the cell deleted at time t , output

$$R_0 - \sum_{i=1}^t \text{cnt}[i].$$

Complexity. $O(\min(n, m)^2 \max(n, m))$ time and $O(nm)$ memory.

Problem C. Random Shuffle

The shuffle is the Fisher–Yates shuffle driven by a xorshift random generator. First recover the values

$$j_i = \text{rand}_i \bmod i + 1$$

from the final permutation: process $i = n, n-1, \dots, 1$, find the current position of value i , set this position to j_i , and undo the swap.

The xorshift generator is linear over $\mathbb{F}_2$. Therefore every output bit of every rand_i is the xor of some subset of the 64 initial bits. The value j_i reveals the lowest $\nu_2(i)$ bits of rand_i , where $\nu_2(i)$ is the exponent of 2 in i . These become linear equations over $\mathbb{F}_2$.

Run Gaussian elimination on the equations. In the worst case $n = 50$, there are at most 17 free variables left. Enumerate all assignments of the free variables, reconstruct the seed, simulate the shuffle, and output any seed that generates the given permutation.

Complexity. $O(n \cdot 64^2 + 2^f n)$, where $f \leq 17$ for the constraints.

Problem D. City Brain

For one route, the best choice is simply a shortest path, and the improvement operations should be distributed as evenly as possible among its edges. For two routes, the only additional feature is that they may share a common path segment. We can assume that all shared edges form one contiguous segment; if two common segments are separated, exchanging the middle subpaths gives an equivalent or better pair with a single common segment.

Let

- c be the number of edges used by both paths;
- d be the number of remaining edges, counted once.

A shared edge contributes twice to the objective, and an exclusive edge contributes once. For a group of s equal edges with coefficient w , spending z improvements on the group is optimal when the improvements are as balanced as possible. If $q = \lfloor z/s \rfloor$ and $r = z \bmod s$, the contribution of this group can be evaluated in $O(1)$.

For fixed (c, d) , split the budget k between the common part and the exclusive part. The resulting one-dimensional function is convex/unimodal on the integers, so an integer ternary search, followed by checking the remaining constant-size interval, gives the best value.

It remains to obtain the possible pairs (c, d) . Run BFS from every vertex, since the graph is unweighted. Enumerate the two endpoints u, v of the shared segment. Then $c = \text{dist}(u, v)$, and the exclusive length is the minimum of the two ways to attach the two required paths to this segment:

$$\begin{aligned} & \text{dist}(s_1, u) + \text{dist}(v, t_1) + \text{dist}(s_2, u) + \text{dist}(v, t_2), \\ & \text{dist}(s_1, u) + \text{dist}(v, t_1) + \text{dist}(s_2, v) + \text{dist}(u, t_2). \end{aligned}$$

Also consider the case $c = 0$, where the two paths are disjoint for the purpose of the cost calculation. For each c , keep the minimum d and evaluate the budget split.

Complexity. $O(n(n + m) + n^2 + n \log k)$ with BFS distances.

Problem E. Tube Master III

The accessible public mirrors I could inspect did not contain a complete official analysis for this problem. The following is the structural reduction that the solution is based on.

At every crossing, the local state is one of seven types: unused, horizontal straight, vertical straight, or one of the four turns. The condition for a cell only depends on the four adjacent crossings and asks that exactly **count** of them are turns. The edge cost is paid by the local states of the two crossings adjacent to that tube.

Thus the task is a weighted constraint-satisfaction problem on the grid graph with a constant-size alphabet. A direct state-compression DP is too large if it stores arbitrary connectivity, but the degree condition “0 or 2” means the chosen tubes form a disjoint union of cycles; no global single-cycle condition is required. This removes the need to distinguish different connected components.

One implementation route is to scan the grid row by row. The frontier state records, for each crossing on the current cut, whether the vertical tube from the previous row is used and whether the crossing has already committed to being a turn or a straight vertex. When the next crossing is processed, enumerate the horizontal and vertical tubes leaving it, reject local degree patterns other than 0 or 2, and once a cell’s fourth corner becomes known, check its required number of turns. Costs of tubes are charged when their two endpoints are determined.

This yields an exact dynamic program over local states; if no final state with all frontier tubes closed survives, the answer is -1 .

Note. The above is a reconstruction-level sketch, not a full low-level implementation of the original hard-problem solution.

Problem F. Rooks

A rook is attacked if the closest rook in the same row or the closest rook in the same column on at least one side belongs to the other player. Only adjacent rooks in the sorted order of a row or column can attack each other; non-adjacent rooks are blocked by the rook between them.

Put all rooks into one list with their owner. Sort by (x, y) to process columns. Consecutive rooks with the same x attack each other exactly when their owners differ; mark both endpoints. Sort by (y, x) and do the same for rows. Count marked rooks of each owner.

Complexity. $O((n + m) \log(n + m))$ time and $O(n + m)$ memory.

Problem G. Prof. Pang’s sequence

Process queries offline in increasing order of their right endpoint. For the current right endpoint r , define a binary value b_l for every $1 \leq l \leq r$:

$$b_l = 1 \iff a_l, a_{l+1}, \dots, a_r \text{ contains an odd number of distinct values.}$$

If the new value is $x = a_r$ and its previous occurrence is at position p , then only starts $l \in [p + 1, r]$ change whether x is present in the suffix, so exactly this interval of b_l is flipped.

For a query $[L, R]$, after processing R we need

$$\sum_{l=L}^R \sum_{t=l}^R [a_l, \dots, a_t \text{ has an odd number of distinct values}].$$

So each time we advance the right endpoint, add the current b_l values to a historical accumulator for all $l \leq r$. Then the answer to $[L, R]$ is the sum of these accumulators over $l \in [L, R]$.

Use a lazy segment tree. A node stores

$$(\text{cnt}_0, \text{cnt}_1, \text{acc}),$$

where cnt_b is the number of positions in the node whose current value is b , and acc is the historical sum. Two range updates are needed:

- flip a range, which swaps cnt_0 and cnt_1 ;
- add the current values to the accumulator, which increases acc by cnt_1 on the updated range.

The lazy tag can be represented by two additive coefficients and one flip bit, or equivalently by the specialized 2×3 affine matrices used in some implementations.

Complexity. $O((n + q) \log n)$ time and $O(n)$ memory.

Problem H. Prof. Pang Earning Aus

The source material accessible from this environment did not include the complete official analysis for this problem. The useful modeling step is as follows.

Let the four buying/exchange operations be represented by non-negative integers:

$$x_{ab}, x_{ac}, x_{bc}, x_{cb},$$

meaning “buy balloons with Aus”, “buy candies with Aus”, “exchange balloons for candies”, and “exchange candies for balloons”. Since selling at the end is always at least as good as keeping remaining goods, all remaining balloons and candies may be assumed to be sold in the final step.

The store-capacity constraints are

$$k_{ab}x_{ab} + k_{cb}x_{cb} \leq n_b, \quad k_{ac}x_{ac} + k_{bc}x_{bc} \leq n_c.$$

The final number of Aus is a linear expression in these four variables after the remaining goods are sold. The difficulty is that the trades must also be orderable without ever making the current amount of Aus, balloons, or candies negative. A feasible optimal solution can be viewed as a bounded integer linear optimization problem with two resource constraints and constant-size coefficients.

Because all six rates are at most 100, the original solution can exploit this constant bound: reduce the search to residue classes of the small coefficients, solve the resulting linear relaxation at its boundary points, and check a constant-size neighborhood around each candidate. This avoids any dependence on n_b or n_c except logarithmic arithmetic on 64-bit or 128-bit integers.

Note. This section preserves the mathematical model, but the exact case analysis of the original hard-problem implementation was not recoverable from the accessible sources.

Problem I. Plants vs Zombies

All zombies move at the same speed. Therefore, among alive zombies, the peashooter always attacks them in increasing order of (t_i, i) : an earlier appearing zombie is farther to the right, and ties are broken by the smaller index.

Sort zombies by this order and process them one by one. Preprocess the spikeweeds by position and prefix sum of their damage. For zombie i , the total spikeweed damage after it has moved for d seconds is the sum of powers of spikeweeds with positions at most dV , which is obtained by binary search in the prefix array.

Maintain S , the number of peas already used to kill earlier zombies in the attack order. For a candidate death time T , the number of peas that have been shot up to then is KT ; hence the current zombie can have received at most

$$\max(0, KT - S)$$

peas by time T . Combine this pea damage with the spikeweed damage and binary-search the earliest T for which the health becomes non-positive. After the death time is found, compute how many peas were actually needed on this zombie and add that to S .

Use 128-bit integers for products such as KT and for accumulated damage.

Complexity. $O((n + m) \log(n + m) \log C)$, where C is the searched time range.

Problem J. Circle

The feasible set of centers is

$$S = \bigcap_{v \in C} \{p \mid |p - v| \leq r\},$$

the intersection of disks of radius r centered at the vertices of the convex polygon. Its boundary consists of circular arcs of radius r , each centered at a polygon vertex that can become tight.

Fix a vertex A . If a covering circle has A on its boundary, then its center lies on the circle of radius r around A . Rotating this center around A eventually gets blocked by one vertex on the left and one on the right. Hence the arc contributed by A is determined by the two neighboring tight vertices in a special cyclic set of vertices.

We can find this cyclic set by pruning. Maintain the current cyclic list of polygon vertices and a queue. For a vertex P with current neighbors L and R , a simple geometric inequality involving $|LR|$ and $\angle LPR$ decides whether there exists a covering circle of radius r with P on the boundary. If not, delete P from the cyclic list and push its two neighbors again. Each vertex is deleted at most once, so the process is linear.

After the remaining tight vertices are known, compute all circular arcs and sum their contribution to the area using Green's formula. If the computed signed area is negative, the feasible set is empty and the answer is 0.

Complexity. $O(n)$ per test case after the input polygon is given, with total $O(\sum n)$.

Problem K. Allin

If Wolf Chicken cannot already form a straight flush from the five known cards, then Daddy Dream can choose hidden cards and future community cards to make a hand that is at least as good. Therefore a straight flush in the known five cards is necessary; a royal flush is immediately sufficient.

For an ordinary straight flush, all five known cards must have the same suit and consecutive ranks. The remaining question is whether the opponent can force a higher straight flush or a royal flush using the three known community cards, the two unrevealed community cards, and his two hole cards. This yields a short case analysis:

- A known royal flush always means **allin**, since the missing player cannot use Wolf's two hole cards to tie it.
- If the known straight flush is not royal and one of Wolf's two hole cards has rank T, J, Q , or K , then no higher straight flush can be completed by the opponent; answer **allin**.
- For the wheel $A2345$, Wolf is safe only when his two hole cards are exactly the endpoints A and 5 of that straight flush.

- Otherwise, if the highest card of Wolf's straight flush is a public card that can be reused by the opponent, the opponent may complete a higher straight flush; answer **check**. The remaining ordinary cases are decided by the same constant-size comparison of the highest card and whether it is private.

Equivalently, one can enumerate the 40 possible straight flushes. Let W be Wolf's known straight flush. If there exists a straight flush not weaker than W that the opponent can still complete without using Wolf's hole cards, then print **check**; otherwise print **allin**.

Complexity. $O(1)$ per test case.

Problem L. Square

Only prime-exponent parity matters. Let v_i be the parity vector of the prime factorization of a_i , and let x_i be the parity vector of t_i . The condition that

$$a_i t_i a_{i+1} t_{i+1}$$

is a square is equivalent to

$$v_i + x_i = v_{i+1} + x_{i+1} \pmod{2}.$$

Thus all vectors $v_i + x_i$ must be equal to one global vector Y .

For each prime p , suppose it appears with odd exponent in exactly c_p of the numbers a_i . If $Y_p = 0$, then p must be inserted into t_i for those c_p positions. If $Y_p = 1$, then it must be inserted for the other $n - c_p$ positions. Choose the cheaper of the two options independently for every prime. Therefore the minimum product is

$$\prod_p p^{\min(c_p, n-c_p)} \pmod{10^9 + 7}.$$

Compute the parity of each a_i by factoring it with a smallest-prime-factor sieve up to 10^6 , and count how many times each prime has odd exponent.

Complexity. $O(A \log \log A + n \log A)$ preprocessing and factorization, where $A = 10^6$.

Problem M. Fillomino

The complete construction from the original Chinese editorial could not be retrieved from the blocked source. The key existence idea is that the toroidal grid is highly connected: with $n, m \geq 3$, deleting a small boundary never separates the board in the way it would on an ordinary rectangle.

A useful way to view the construction is through a Hamiltonian cycle of the toroidal grid. If we can find such a cycle and cut it into three consecutive arcs of lengths

$$\text{cnt}_1, \quad \text{cnt}_2, \quad \text{cnt}_3,$$

so that the i -th arc contains the i -th son's cell, then assigning these three arcs to A, B, C gives a valid output immediately, because each arc is connected along the cycle.

The specialized construction used for this problem starts from a row-snake or column-snake Hamiltonian cycle on the torus, tries the constant number of rotations, reversals, and axis swaps, and uses local 2×3 reroutings to adjust the cyclic order and distances of the three marked cells. Since both dimensions are at least 3, these local reroutings do not destroy Hamiltonicity. After the cycle has been adjusted, print the three consecutive arcs.

Note. This section records the verified high-level construction strategy, but not every low-level rerouting case.